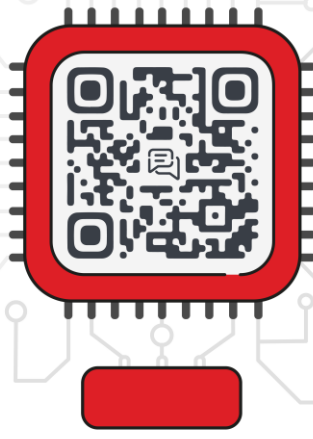
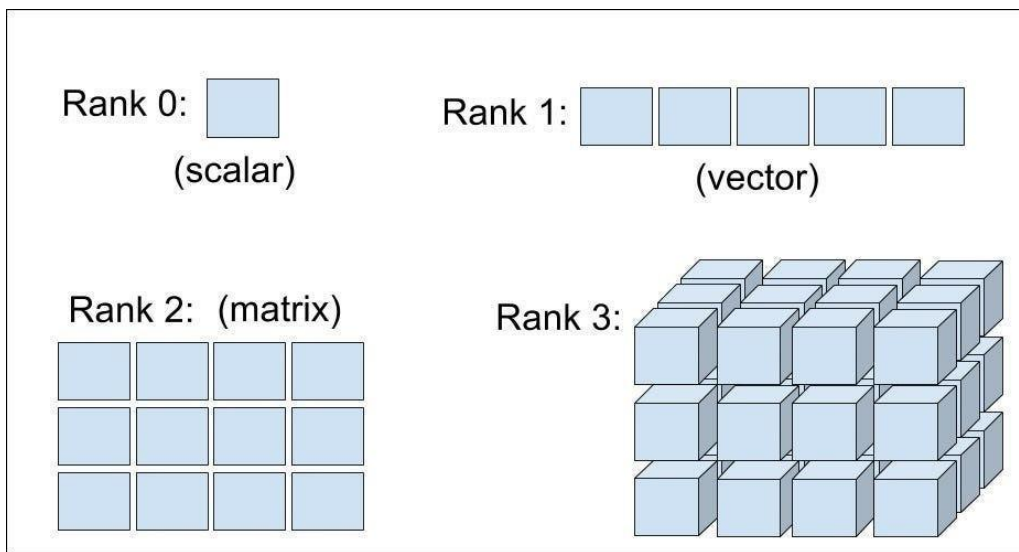




الموتر Tensor

- هو مصفوفة متعددة الأبعاد تمثل البيانات الأساسية في PyTorch ويشبه مصفوفات NumPy لكنه يدعم عمليات متقدمة مثل الحسابات التلقائية للتدرجات
- يستخدم لتخزين ومعالجة كل من :
 1. المدخلات (Inputs): البيانات التي نعطيها للنموذج (مثل الصور)
 2. المخرجات (Outputs): النتائج التي ينتجها النموذج
 3. معاملات النموذج (Parameters): الأوزان والقيم التي يتعلمها النموذج أثناء التدريب
- على سبيل المثال:

يمكن اعتبار تينسور أحادي البعد كمتجه، وثنائي البعد كمصفوفة هذا الهيكل يسمح بإنشاء علاقات متعددة الخطوط بين الكائنات الجبرية في فضاء متجهي (بمعنى يربط بين أرقام أو متجهات متعددة بطريقة خطية منظمة بفضاء متعدد الأبعاد لأجل تسهيل الحسابات)



طرق إنشاء الموتر :

يمكن إنشاء الموترات من قوائم Python أو مصفوفات NumPy

▪ مثال 1 : إنشاء موتر من قائمة

```
--
13 mylist = [1,2,3,4]
14 mytensor = torch.tensor(mylist)
15 print(mytensor)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL POF

```
tensor([1, 2, 3, 4])
```

▪ مثال 2 : من مصفوفة

```
17 myarr = np.array([[1,2],[3,4]])
18 mytensor_2 = torch.from_numpy(myarr)
19 print(mytensor_2)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
tensor([[1, 2],
        [3, 4]])
```

عند إنشاء موتر من مصفوفة NumPy لا يتم تخصيص موقع ذاكرة خاص به وإنما تشترك كل من المصفوفة والموتر في نفس موقع الذاكرة بحيث إذا قمت بإجراء أي تغيير على الموتر يتم عكسه على المصفوفة الأصلية التي تم إنشاؤه منها

▪ مثال : تغيير ع الموتر السابق (تغيير العنصر ذات الاندكس (1,1) من 4 ل 5

```
--
17 myarr = np.array([[1,2],[3,4]])
18 mytensor_2 = torch.from_numpy(myarr)
19
20 mytensor_2[1,1]=5
21 print(myarr)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

```
[[1 2]
 [3 5]]
```

أيضاً يمكننا استخدام `mytensor_2.numpy()` من أجل تحويل الموتر إلى مصفوفة NumPy والبيانات سوف تكون مشتركة في الذاكرة (يعني أي تغيير في واحد يؤثر على الثاني) ومثل مصفوفات NumPy الموترات في PyTorch متجانسة يعني كل العناصر داخلها يجب أن تكون ذات نوع بيانات واحد (مثل كلها أرقام عشرية أو كلها أعداد صحيحة وليس مختلطة)

■ مثال: لدينا الموتر `[2.5, 1.5]` عناصره من نوع `floats` لا يمكننا إضافة رقم 3 (`integer`) مباشرة بدون تحويله، من أجل التجانس

أيضاً لدينا طرق أخرى لإنشاء الموترات :

■ مثال 1 : انشاء موتر جميع قيم عناصره اصفار، الطريقة المشابهة من المصفوفات `np.zeros(3,3)` حيث الموتر هو الوحدة الأساسية لتمثيل البيانات في بايتورتش حيث الموتر :

- يتعلم بنفسه (يحتفظ بمعلومات التدرج `gradient`)
- يعمل على كرت الشاشة (GPU) لتسريع العمليات
- يتتبع كل العمليات التي مر بها (للعودة للخلف في التدريب)
- أيضاً هو اللغة الوحيدة التي يتحدث بها بايتورتش
- فكما الكمبيوتر يفهم فقط 0 و 1
- يفهم فقط الموترات PyTorch

```
23 z=torch.zeros((2,3))
24 print(z)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

■ مثال 2 : انشاء موتر قيم عناصره = 1 , موتر قيم عناصره عشوائية , موتر خصائصه مشابهة لموتر آخر

```
26 x=torch.ones((2,3))
27 print('Ones : ' , x)
28
29 y=torch.rand((2,3))
30 print('Random : ',y)
31
32 c=torch.ones_like(mytensor_2)
33 print('Ones_like : ' ,c)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUP

```
Ones : tensor([[1., 1., 1.],
               [1., 1., 1.]])
Random : tensor([[0.2132, 0.8138, 0.6506],
                 [0.3620, 0.4469, 0.7726]])
Ones_like : tensor([[1, 1],
                    [1, 1]])
```

عمليات الموترات في PyTorch

يمكننا إجراء العديد من العمليات الحسابية على موترات PyTorch بنفس الطريقة التي نتعامل بها مع مصفوفات NumPy لكن قدراتها أحيانا تكون أكثر

1. العمليات الحسابية : عملية حسابية مع الأرقام الفردية (Broadcasting) او عملية بين موترين

عندما نقوم بعملية حسابية (مثل الجمع أو الضرب) بين موتر ورقم واحد (Scalar) فإن بايتورش تقوم تلقائيا بتطبيق هذه العملية على كل عنصر في الموتر (Broadcasting)

■ امثلة:

```
35 myarr = np.array([[1.0,2.0],[3.0,4.0]])
36 tensor1 = torch.from_numpy(myarr)
37 a=tensor1+1
38 print(a)
39
40 s=tensor1/ tensor1
41 print(s)
42
43 d=tensor1.sin()
44 print(d)
45
46 f=tensor1.cos()
47 print(f)
48
49 g=tensor1.sqrt()
50 print(g)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITLENS

```
tensor([[2., 3.],
        [4., 5.]], dtype=torch.float64)
tensor([[1., 1.],
        [1., 1.]], dtype=torch.float64)
tensor([[1., 1.],
        [1., 1.]], dtype=torch.float64)
tensor([[ 0.5403, -0.4161],
        [-0.9900, -0.6536]], dtype=torch.float64)
tensor([[1.0000, 1.4142],
        [1.7321, 2.0000]], dtype=torch.float64)
```

في المثال الاول تم إضافة الرقم 1 إلى كل عنصر في الموتر:

■ $1 + 1 = 2$

■ $2 + 1 = 3$

■ $3 + 1 = 4$

■ $4 + 1 = 5$

حيث يتم تطبيق نفس المبدأ على جميع العمليات الأخرى مثل الطرح والضرب والقسمة في المثال الثاني عملية بين موترين تم تقسيم الموتر على نفسه حيث تمت قسمة كل عنصر على العنصر المقابل له في نفس الموضع:

$$\begin{aligned} 1.0 / 1.0 &= 1.0 \\ 2.0 / 2.0 &= 1.0 \\ 3.0 / 3.0 &= 1.0 \\ 4.0 / 4.0 &= 1.0 \end{aligned}$$

ملاحظة : عندما نقوم بعملية حسابية بين موترين فإن بايتورث تطبيق العملية عنصراً بعنصر (element-wise) بشرط أن يكونا بنفس الأبعاد (نفس عدد الصفوف ونفس عدد الأعمدة)

- في المثال الثالث تم تطبيق الدالة ساين على كل عنصر
- في المثال الرابع تم تطبيق الدالة كوساين على كل عنصر
- في المثال الخامس تم حساب الجذر التربيعي لكل عنصر

تعميم :

- مع رقم واحد (Scalar): العملية تُطبق على كل عناصر الموتر تلقائياً (Broadcasting)
- مع موتر آخر : العملية تُطبق عنصر بعنصر بشرط أن يكون للموترين نفس الشكل

2. عمليات احصائية :

■ امثلة :

```
--
56 mean, median, min_val, max_val = tensor1.mean(), tensor1.median(), tensor1.min(), tensor1.max()
57 print ("Statistical Quantities: ")
58 print ("Mean: {}, \nMedian: {}, \nMinimum: {}, \nMaximum: {}".format(mean, median, min_val, max_val))
59 print ("The 50-quantile is present at {}".format(tensor1.quantile(0.5)))
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITLENS

```
Statistical Quantities:
Mean: 2.5,
Median: 2.0,
Minimum: 1.0,
Maximum: 4.0
The 50-quantile is present at 2.5
```

تم حساب كل من :

1. المتوسط الحسابي (Mean) : مجموع كل الأرقام مقسوماً على عدده
2. الوسيط (Median) : الرقم الذي يقع في المنتصف تماماً عندما نرتب الأرقام من الأصغر إلى الأكبر
3. القيمة الدنيا (Minimum) : أصغر رقم
4. القيمة العليا (Maximum) : أكبر رقم
5. الكمي (Quantile) : هو رقم يعطي أي نسبة من الأرقام أصغر من هذا الرقم

- أيضاً يمكننا دمج الموترات (Concatenation) عن طريق `torch.cat()` ولكن يجب ان نحدد اتجاه الدمج ولدينا نوعين
1. الدمج الرأسى (Vertical Concatenation) عندما نستخدم `axis=0` فإن بايتورش يضع الموتر الثاني أسفل الموتر الأول أي يكون الموتر الثاني كصفوف جديدة في الموتر الاول
 2. الدمج الأفقي (Horizontal Concatenation) عندما نستخدم `axis=1` فإن بايتورش يضع الموتر الثاني بجانب الموتر الأول أي يكون الموتر الثاني كأعمدة جديدة في الموتر الاول
- مثال :

```
52 tensor2 = torch.tensor([[5,6],[7,8]])
53 p=torch.cat([tensor1, tensor2], 0)
54 print('axis=0 :' , p)
55 u=torch.cat([tensor1, tensor2], 1)
56 print('axis=1 :' , u)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITL

```
axis=0 : tensor([[1., 2.],
                 [3., 4.],
                 [5., 6.],
                 [7., 8.]], dtype=torch.float64)
axis=1 : tensor([[1., 2., 5., 6.],
                 [3., 4., 7., 8.]], dtype=torch.float64)
```

- أيضاً يتوفر دوال اختصار للدمج مثل `hstack` و `vstack` :

■ `torch.vstack()` اختصار لـ Vertical Stack هي نفسها `torch.cat(..., axis=0)`

■ `torch.hstack()` اختصار لـ Horizontal Stack هي نفسها `torch.cat(..., axis=1)`

فبدلاً من كتابة `axis=0` يمكننا ببساطة كتابة `vstack` وبدلاً من `axis=1` نكتب `hstack`

- تغيير شكل الموتر (Reshape) :

■ تحويل الموتر إلى صف واحد وعدد غير محدد من الأعمدة حيث بايتورش يتولى معرفة عدد العناصر بنفسه مثل

■ `(-1, 1)` حيث :

■ 1 يعني صف واحد و -1 يعني ان بايتورش يكتشف بنفسه عدد الأعمدة بناءً على عدد العناصر

■ و `(1, -1)` حيث :

■ -1 يعني عدد غير محدد من الاسطر حيث بايتورش يكتشف بنفسه عدد الاسطر بناءً على عدد العناصر و 1:

عمود واحد

■ `(4,)` : تحويل الى متجه ب 4 عناصر

■ مثال:

```

8 q=torch.hstack((tensor1,tensor2))
9 print('hstack : ', q)
10 w=torch.vstack((tensor1,tensor2))
11 print('vstack : ',w)
12 e=torch.reshape(tensor1, (1, -1))
13 print('One row : ',e)
14 r=torch.reshape(tensor1, (4,))
15 print('Vector : ',r)
16 t=torch.reshape(tensor1, (-1, 1))
17 print('One column : ',t)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```

hstack : tensor([[1., 2., 5., 6.],
                [3., 4., 7., 8.]], dtype=torch.float64)
vstack : tensor([[1., 2.],
                [3., 4.],
                [5., 6.],
                [7., 8.]], dtype=torch.float64)
One row : tensor([[1., 2., 3., 4.]], dtype=torch.float64)
Vector : tensor([1., 2., 3., 4.], dtype=torch.float64)
One column : tensor([[1.],
                    [2.],
                    [3.],
                    [4.]], dtype=torch.float64)

```

الميزات الرئيسية ل بايتورش :

لقد تطوّر بايتورش ليصبح أحد الإطارين (frameworks) الأكثر شيوعاً للتعلم العميق وذلك نسبة لمرونته العالية لاستخدامه للرسوم البيانية الحسابية الديناميكية

ماذا يعني هذا؟

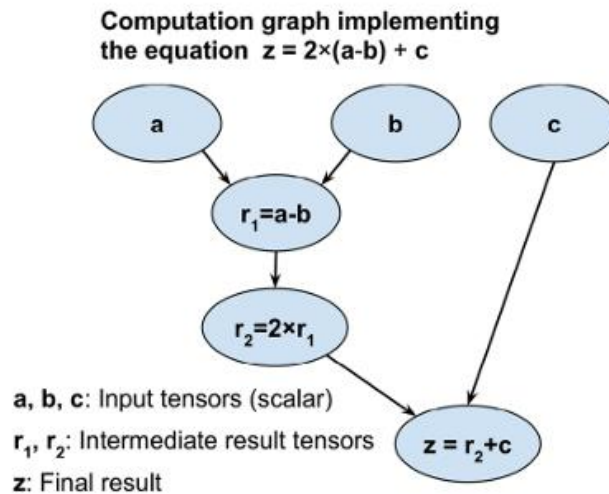
بدلاً من تخطيط لبناء النموذج كله مرة واحدة ثم تجربته، يتيح لك بايتورش أن تبنيه خطوة بخطوة وتتحقق من كل جزء والميزة الأخرى هي سهولة تصحيح الأخطاء :

هذه الطريقة الديناميكية تجعل اكتشاف الأخطاء وإصلاحها سهلاً جداً حيث يمكنك تشغيل الكود سطر بسطر والتحقق من قيم كل المتغيرات بأي لحظة هذا يعطي تحكم كامل وفهم عميق لما يحدث داخل النموذج حيث هذه المرونة والقدرة على التصحيح الفوري تجعل عملية تطوير وتدريب الشبكات العصبية في بايتورش عملية بسيطة وسلسة

الرسوم البيانية الحسابية في بايتورث :

بايتورث يعالج أي عملية حسابية خطوة بخطوة بحيث كل خطوة مبنية على ما قبلها وباتجاه واحد فقط من البداية الى النهاية وغير دورية يعني الخطوة لا تعود لنقطة البداية أبدا وتضمن انو الحسابات منطقية وواضحة اذا بايتورث تستخدم الرسم البياني لتوضيح العلاقة بين كل الأرقام (الموترات) والعمليات الحسابية من البداية (المدخلات) إلى النهاية (المخرجات)

▪ مثال : $z = 2 \times (a - b) + c$



1. البداية : يأخذ المدخلات a و b و c كنقاط بداية وهي اعداد فردية
 2. الخطوة الأولى : يرسم عملية الطرح ($a - b$) وينتج قيمة جديدة لنسميها مثلا r_1
 3. الخطوة الثانية : يأخذ r_1 ويرسم عملية الضرب في ($r_1 \times 2$) ويعطي r_2
 4. الخطوة الثالثة : يأخذ r_2 والمدخل c ويرسم عملية الجمع ($r_2 + c$) ليعطي النتيجة النهائية z
- لنرى الدالة الموافقة للمثال :

```
def compute_z(a, b, c):
    r1 = torch.sub(a, b)
    r2 = torch.mul(r1, 2)
    z = torch.add(r2, c)
    return z
```

لنرى الان تنفيذ الدالة بأنواع مختلفة من المدخلات :

```
>/
58 print('Scalar Inputs:', compute_z(torch.tensor(1), torch.tensor(2), torch.tensor(3)))
59 print('Rank 1 Inputs:', compute_z(torch.tensor([1]), torch.tensor([2]), torch.tensor([3])))
60 print('Rank 2 Inputs:', compute_z(torch.tensor([[1]]), torch.tensor([[2]]), torch.tensor([[3]])))
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITLENS

Scalar Inputs: tensor(1)
 Rank 1 Inputs: tensor([1])
 Rank 2 Inputs: tensor([[1]])

في النوع الاول استخدمنا ارقام فردية scalar

النوع الثاني متجهات من قيمة واحدة والناتج ايضا قيمة واحدة

النوع الثالث مصفوفات بسطر واحد وعمود واحد والنتيجة أيضاً من نفس البعد

الآن الموترات لتخزين وتحديث معاملات النموذج:

لتدريب نموذج يجب أن يعرف كيف يُحسّن نفسه حيث يفعل ذلك عن طريق تغيير أرقامه الداخلية (المعاملات) تدريجياً في بايتورث نستخدم موترات (Tensors) خاصة لتخزين وتحديث قيم النموذج (Parameters) أثناء التدريب و لتمكين حساب التدرجات (Gradients) اللازمة للتحديث نقوم بضبط الخاصية requires_grad=True

- ملاحظة: لا يمكن استخدام الخاصية requires_grad لحساب التدرجات إلا مع الموترات ذات الأنواع العشرية أو المركبة

■ مثال :

```
59 a = torch.tensor(3.14, requires_grad=True)
60 print(a)
61 b = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
62 print(b)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITLENS

```
tensor(3.1400, requires_grad=True)
tensor([1., 2., 3.], requires_grad=True)
```

■ ملاحظة 1 :

- requires_grad يتم تعيينه إلى False بشكل افتراضي حيث يمكن تعيين هذه القيمة True عن طريق requires_grad_()

■ ملاحظة 2:

- In-place تعني أن العملية يتم تنفيذها مباشرة على البيانات الأصلية دون الحاجة لإنشاء نسخة جديدة منها في الذاكرة
- بدلاً من أخذ قيمة x وإجراء عملية عليها ثم تخزين النتيجة في مكان جديد تقوم الدالة بتعديل محتوى x في مكانه الأصلي
- في بايتورث تنتهي هذه الدوال دائماً بـ _ (مثل add_() أو zero_())

■ مثال :

```
>8
59 w = torch.tensor([1.0, 2.0, 3.0])
60 print(w.requires_grad)
61 w.requires_grad_()
62 print(w.requires_grad)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JI

```
False
True
```

تم انشاء موتر بقيم عشرية ولكن بايتورش لا يقوم بتتبع العمليات الحسابية لإن requires_grad تم أخذه بشكل افتراضي False وبالتالي لا يتم حساب التدرجات .

أما عندما استخدمنا w.requires_grad_() لاحظ أن requires_grad_() تنتهي ب_ وهي الموافقة لفكرة In-place إذ سيتم تعديل الموتر نفسه وتغيير requires_grad_() إلى True إذا بايتورش تقوم بتتبع العمليات الحسابية لتتمكن لاحقاً من حساب التدرجات عند الحاجة .

حساب التدرجات عبر التفاضل التلقائي automatic differentiation

كما تحدثنا في المحاضرات السابقة تعتمد عملية تحسين الشبكات العصبية (Optimization) على حساب تدرجات الخسارة (Gradients of the loss) بالنسبة لأوزان النموذج حيث هذه التدرجات هي التي تستخدمها خوارزمية (SGD) لمعرفة الاتجاه الذي يجب أن تُعدّل به الأوزان لتحسين أداء النموذج

في بايتورش :

1. **المرور الأمامي (Forward Pass)** : نقوم بحساب قيمة التنبؤ z ثم حساب قيمة الخسارة loss التي تقيس مدى بُعد التنبؤ عن الهدف الحقيقي y
2. **المرور الخلفي (Backward Pass)** : عن طريق استدعاء الدالة loss.backward() يقوم بايتورش تلقائياً بحساب التدرجات لكل المتغيرات التي تم تفعيل فيها خاصية
3. **النتيجة** : يتم تخزين قيم التدرجات الناتجة $(\frac{dL}{dw})$ و $(\frac{dL}{db})$ داخل (grad) لكل موتر

■ مثال :

```
58 w = torch.tensor(1.0, requires_grad=True)
59 b = torch.tensor(0.5, requires_grad=True)
60 x = torch.tensor([1.4])
61 y = torch.tensor([2.1])
62 z = torch.add(torch.mul(w, x), b)
63 loss = (y-z).pow(2).sum()
64 loss.backward()
65 print('dL/dw : ', w.grad)
66 print('dL/db : ', b.grad)
67
68 # verifying the computed gradient
69 print(2 * x * ((w * x + b) - y))
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
dL/dw : tensor(-0.5600)
dL/db : tensor(-0.4000)
tensor([-0.5600], grad_fn=<MulBackward0>)
```

قمنا بحساب الخسارة من خلال:

$$\text{Loss} = (y - (w * x + b))^2$$

حيث z يمثل المرور الأمامي وتم بدء المرور الخلفي من خلال loss.backward() حيث يبدأ من loss إلى z إلى w و b

للتأكد من حساب التدرج :

نريد حساب تدرج دالة الخسارة L بالنسبة ل w أي: $\frac{dL}{dw}$

لدينا :

$$Z = w \cdot x + b \quad \text{معادلة التنبؤ}$$

$$L = (y - z)^2 \quad \text{دالة الخسارة}$$

كما تعلمنا سابقاً لا يمكننا حساب تدرج L بالنسبة لـ w بشكل مباشر إذاً نحسبه عن طريق Z وحساب Z عن طريق w

$$\frac{dl}{dw} = \frac{dl}{dz} \cdot \frac{dz}{dw}$$

أولاً : نحسب $\frac{dl}{dz}$

$$L = (y - z)^2$$

$$l' = -2(y - z)$$

ثانياً : نحسب $\frac{dz}{dw}$ باعتبار كل من x و b ثابت

مشتق z بالنسبة لـ w هو x

مشتق b بالنسبة لـ w هو 0

$$\frac{dz}{dw} = x$$

ثم نضرب النتيجة :

$$\frac{dl}{dw} = \frac{dl}{dz} \cdot \frac{dz}{dw} = -2x(y - z)$$

وهذا الموافق للكود :

`Print(2 * X * ((w * x + b) - y))` ضربنا ناتج الضرب اليدوي بناقص

باعتبار : $y=2.1, x=1.4, b=0.5, w=1$

$$\frac{dl}{dw} = -2 * 1.4 * (2.1 - 1.9) = -0.56$$
 بالتعويض :

من المرور الأمامي : $z = (1 * 1.4 + 0.5) = 1.9$

حيث نرى أن الناتج مماثل لنتيجة بايتورث بنفس الطريقة يتم حساب $\frac{dl}{db}$

ملاحظة:

- نستخدم الخاصية `grad_fn` لمعرفة العملية الحسابية التي أدت إلى انشاء الموتر
- ما هي خاصية `grad_fn`؟
- هي إشارة مرجعية (Reference) للعملية الرياضية التي أنشأت الموتر حيث تعتبر هذه الخاصية عقدة (Node) في مخطط الحسابات (Computational Graph) وهي الطريق الذي يتبعه بايتورث للرجوع إلى الخلف أثناء عملية المرور العكسي (Backpropagation) لحساب التدرجات

القواعد الأساسية (Key Takeaways) :

1. Leaf Tensors :

- هي الموتر الذي ينشئه المستخدم يدوياً (مثل الأوزان w والانحياز b)
- هذا الموتر لا ينتج عن عملية حسابية لذا فإن `grad_fn` لها يكون `None`

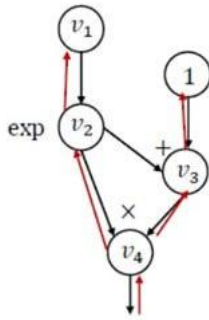
```
w.grad_fn == None
B.grad_fn == None
```

2. الموتر الناتج عن عمليات (Calculated Tensors) :

- أي موتر ينتج عن عملية مثل MulBackward0 سيكون له grad_fn
- حيث يتم إنشاء هذا الموتر بواسطة عملية ضرب بايتورش يعرف كيفية الرجوع العكسي من خلاله
- مثل : $z = w * x + b$ حيث تم توجه بايتورش بأن لاشتقاق z يجب عليك العودة إلى عمليتي الضرب والجمع

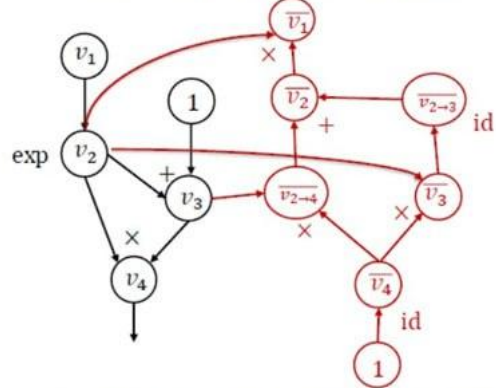
Reverse mode AD vs Backprop

Backprop



- Run backward operations the same forward graph
- Used in first generation deep learning frameworks (caffe, cuda-convnet)

Reverse mode AD by extending computational graph



- Construct separate graph nodes for adjoints
- Used by modern deep learning frameworks

AD تعني Automatic Differentiation التفاضل الآلي

- Reverse mode يعني أننا نحسب المشتقات بالرجوع للوراء (من الناتج إلى المدخلات)، وهذه الطريقة التي سوف تستخدمها TensorFlow و PyTorch
- السبب؟ لأنه في الشبكات العصبية لدينا خرج واحد فقط (Loss) ملايين من الدخول مدخل (الأوزان) حساب المشتقات بالرجوع للوراء يسمح لنا بحساب جميع المشتقات (dL/dw_1 , dL/dw_2 , ...) في مرور واحد فقط للخلف وهذا ما يميزها

1. مخطط الـ Backprop المرور الخلفي

- نستخدم نفس الرسم البياني الذي استخدمناه في المرور الأمامي لكي نحسب المشتقات أي لا نرسم مخطط مساعد حيث يستخدم في أول جيل من أطر التعلم العميق (caffe, cuda-convnet) حيث تعتبر طريقة قديمة

■ كيف يعمل؟

الحساب الأمامي (Forward Pass)

- نبدأ من المدخلات ونحسب كل شيء حتى نصل للناتج النهائي (v_4) كما تعلمنا
- v_1 هو الدخل
- $v_2 = \exp(v_1)$ دالة الاس
- $v_3 = v_2 + 1$
- $v_4 = v_2 * v_3$

الحساب الخلفي (Backward Pass) حساب المشتقات

- نبدأ من v_4 ونتحرك للوراء هدفنا هو حساب المشتق الجزئي للخطأ بالنسبة لكل متغير اي

$$\left(\frac{dv_4}{dv_1}, \frac{dv_4}{dv_2}, \frac{dv_4}{dv_3} \right)$$

■ الخطوة الأولى: v_4 هو الخطأ (Loss)

نبدأ بـ $dv_4/dv_4 = 1$ (مشتق أي شيء بالنسبة لنفسه هو 1)

■ الخطوة الثانية: حساب $\frac{dv_4}{dv_2}$ و $\frac{dv_4}{dv_3}$

بما أن $v_4 = v_2 * v_3$ فباستخدام قاعدة الضرب:

$$dv_4/dv_2 = v_3$$

$$dv_4/dv_3 = v_2$$

■ الخطوة الثالثة: حساب $\frac{dv_4}{dv_1}$

باستخدام قاعدة السلسلة (Chain Rule) التي تحدثنا عنها سابقا

$$\frac{dv_4}{dv_1} = \left(\frac{dv_4}{dv_2} \right) * \left(\frac{dv_2}{dv_1} \right)$$

$$\frac{dv_4}{dv_2} = v_3 \text{ نعرف أن}$$

نعرف أن $v_2 = \exp(v_1)$ وبالتالي v_2 $\frac{dv_2}{dv_1} = \exp(v_1)$ إذاً

$$\frac{dv_4}{dv_1} = v_3 * v_2$$

■ المشكلة: يعتمد على إعادة استخدام نفس الرسم البياني من اجل حساب المشتقات وهذا قد يكون محدود أو غير فعال في بعض الحالات المعقدة

2. مخطط الـ Reverse mode AD by extending computational graph التفاضل الآلي بالوضع

العكسي عبر توسيع الرسم البياني

- الوصف: هذه الطريقة الحديثة المستخدمة في PyTorch و TensorFlow لا يعيد استخدام الرسم البياني الأصلي بل يبني رسماً بيانياً جديد خاص بحساب المشتقات

الممر الأمامي (Forward Pass)

نفس الخطوات السابقة:

$$v_1 \rightarrow v_2 = \exp(v_1) \rightarrow v_3 = v_2 + 1 \rightarrow v_4 = v_2 * v_3$$

بناء الرسم البياني الجديد لحساب المشتقات

بعد انتهاء الممر الأمامي نبدأ ببناء الرسم البياني الجديد هذا الرسم البياني يتكون من عقد جديدة تمثل المرافق (Adjoint) لكل عقدة في الرسم البياني الأصلي

نبدأ من $\bar{v}_4$ وهو مشتق الخطأ بالنسبة لـ v_4 ، ونضع له قيمة 1 لأننا نحسب مشتق الخطأ بالنسبة لنفسه نتحرك للوراء ونحسب المشتقات باستخدام قاعدة السلسلة

الممر الخلفي (Backward Pass)

1. الخطوة الأولى: حساب $\bar{v}_3$ و $\bar{v}_2$ من $\bar{v}_4$ (ملاحظة $\bar{v}_2$ تقرأ مشتق الخطأ بالنسبة لـ v_3)
بما أن $v_4 = v_2 * v_3$ فإن:

$$\bar{v}_2 += \bar{v}_4 * v_3$$

($\frac{dv_4}{dv_2} = v_3$) حيث v_4 يعتمد على v_2 من خلال السهم الذي عليه اشارة ضرب
عند حساب $\bar{v}_2$ مشتق الخطأ بالنسبة لـ v_2 نجمع تأثير v_2 من خلال كل سهم داخل عليها :

$$\begin{aligned} \bar{v}_2 += \bar{v}_3 * \left(\frac{dv_3}{dv_2}\right) & \bullet \text{ وهذا السهم القادم من } v_3 \rightarrow \bar{v}_2 \\ \bar{v}_2 += \bar{v}_4 * \left(\frac{dv_4}{dv_2}\right) & \bullet \text{ وهذا السهم القادم من } v_4 \rightarrow \bar{v}_2 \\ \bar{v}_3 += \bar{v}_4 * v_2 & \end{aligned}$$

($\frac{dv_4}{dv_3} = v_2$) حيث v_4 يعتمد على v_3 من خلال السهم الذي عليه اشارة ضرب

2. الخطوة الثانية: حساب $\bar{v}_2$ من $\bar{v}_3$

بما أن $v_3 = v_2 + 1$ ، فإن:

$$\bar{v}_2 += \bar{v}_3 * 1$$

($\frac{dv_3}{dv_2} = 1$) حيث v_3 يعتمد على v_2 من خلال السهم الذي عليه اشارة جمع
إذاً $\bar{v}_2$ الآن يساوي $\bar{v}_3 * 1 + \bar{v}_4 * v_3$

3. الخطوة الثالثة: حساب $\bar{v}_1$ من $\bar{v}_2$

بما أن $v_2 = \exp(v_1)$ فإن:

$$\frac{dv_2}{dv_1} = \exp(v_1) = v_2 \text{ لأن } \bar{v}_1 += \bar{v}_2 * v_2 \text{ مشتق دالة الاس هو نفسها}$$

إذاً $\bar{v}_1$ يساوي $(\bar{v}_4 * v_3 + \bar{v}_3 * 1) * v_2$

ملاحظة : $\bar{v}_2 \rightarrow \bar{v}_3$ تعني مشتق v_3 بالنسبة لـ v_2 اي $\frac{dv_3}{dv_2}$

$$\frac{dv_4}{dv_2} \text{ اي } v_2 \text{ بالنسبة لـ } v_4 \text{ مشتق } \bar{v}_2 \rightarrow_4$$

مقارنة

الميزة	Reverse mode AD	Backprop
الرسم البياني	يرسم مخطط منفصل (أو يوسعه) لحساب المشتقات	يستخدم نفس الرسم البياني للمرور الامامي والخلفي
المرونة	أكثر مرونة وفعالية خاصة مع العمليات الديناميكية	أقل مرونة محدود في بعض الحالات
الاستخدام	الإطارات الحديثة (PyTorch, TensorFlow)	الإطارات القديمة (Caffe)

لماذا بايتورش يستخدم طريقة التراكم العكسي reverse accumulation ؟

بسبب كفاءته العالية لتعامله مع النماذج الضخمة لنرى المناقشة التالية :

1. في التعلم العميق (Deep Learning) :

- المدخلات: ملايين المعاملات (Weights الأوزان) (w1, w2, w3,...)
- المخرجات: قيمة واحدة فقط (الخسارة Loss) (Scalar)
- المطلوب: حساب تأثير كل وزن من هذه الملايين على هذه الخسارة الواحدة (dL/dw)

2. Forward Mode تكلفة عالية

- يلزمنا مرور واحد (Forward Pass) لكل متغير على حدة
- ففي حال كان لدينا مليون وزن نحن بحاجة لمليون عملية حسابية للحصول على جميع الاشتقاقات
- هذا يجعل التدريب مستحيلاً من الناحية الزمنية والحسابية

3. Reverse Mode كفاءة عالية

- مرور واحد فقط (One Backward Pass) عن طريق loss.backward() يتم لعودة من الخرج (Loss) إلى الخلف (المدخلات)
- النتيجة: مرور عكسي واحد يكفي لحساب تدرجات جميع الأوزان (المعاملات)
- إذاً لهذا السبب تعتمد بايتورش Reverse-mode AD لأنه يجعل تدريب النماذج التي تحتوي على مليارات المعاملات سريع

خلاصة:

يعود اعتماد بايتورث على Reverse Accumulation إلى طبيعة الشبكات العصبية التي تتكون من ملايين المتغيرات مقابل خرج واحد للخطأ بينما يتطلب Forward mode عملية حسابية لكل متغير و يتيح Reverse mode استخراج حساب جميع التدرجات عبر مرور واحد فقط وهذا يجعل عملية التدريب قابلة للتنفيذ وسريعة .

■ ملاحظة: يعد استخدام طريقة Forward mode مثالي في حال كانت المدخلات اقل من المخرجات

تقريب تايلور (Taylor Approximation) وعلاقته بـ طريقة نيوتن (Newton's Method) :

تعتبر طريقة نيوتن من الخوارزميات المتقدمة في التحسين (Optimization) حيث لا تكتفي بميل المنحنى فقط بل تدرس انحناءه أيضاً لتصل إلى الحل بشكل أسرع عند كل خطوة، تقوم طريقة نيوتن بتقريب الدالة $f'(x)$ حول النقطة الحالية x_k باستخدام كثير حدود تايلور من الدرجة الثانية (Quadratic Approximation) بدلاً من التعامل مع الدالة الأصلية نتعامل مع منحنى تربيعي بشكل حرف U يسهل إيجاد قمته أو قاعه .

a. في حال كانت الدوال احادية البعد اي كتغير واحد (Univariate) :

يتم إيجاد النقطة التالية $(x_k + 1)$ باستخدام المشتق الأول والثاني:
القانون :

$$X_{k+1} = X_k - \frac{f'(X_k)}{f''(X_k)}$$

■ $f'(x_k)$: المشتق الأول (الميل Slope)

■ $f''(x_k)$: المشتق الثاني (الانحناء Curvature)

b. أما في حال كانت الدوال متعددة الابعاد اي متعددة المتغيرات (Multivariate) :

هنا في معادلتنا سنستخدم المصفوفات

$$X_{k+1} = X_k - H(X_k)^{-1} * f'(X_k)$$

■ $H(X_k)$ هي مصفوفة هيسيان (Hessian Matrix) والتي تحتوي على المشتقات الثانية لجميع المتغيرات

■ $H(X_k)^{-1}$ مقلوب مصفوفة هيسيان

العيوب والتحديات (Drawbacks) :

■ **التكلفة الحسابية العالية:** حساب مقلوب المصفوفة (Inverting the Hessian) وخاصة عندما يكون لدينا

ملايين المتغيرات كما في الشبكات العصبية تكون التكلفة عالية

- المشكلات الموضعية: في الدوال غير المحدبة (Non-convex) قد تنحبس (تعلق) الخوارزمية في نهايات صغرى محلية (Local Minima) أو نقاط سرجية (Saddle Points) الذين تحدثنا عنهم سابقاً بدلاً من الوصول للحل الأمثل global

متى نستخدمها؟

- في تعلم الآلة (ML): نستخدم عندما تكون الدالة محدبة (Convex) وعدد المتغيرات مناسب
- في التعلم العميق (DL): لا نستخدم وذلك يعود لضخامة عدد المعاملات (Parameters) وطبيعة الدوال غير المحدبة التي تجعل حساب مصفوفة هيسيان صعب

خلاصة:

تتفوق طريقة نيوتن على الطرق التقليدية وذلك لقدرتها على استخدام الانحناء عبر تقريب تايلور مما يجعلها تصل للهدف بخطوات أقل ولكن مع ذلك تبقى هناك عوائق التكلفة الحسابية وصعوبة التعامل مع الدوال غير المحدبة .

حل مشكلة تصنيف XOR

هل تذكر مشكلة XOR التي تحدثنا عنها سابقاً حيث كانت مشكلة تصنيف لا يمكن حلها باستخدام خط مستقيم واحد (Linear Separability) يفصل بين النقاط لذا سنستخدم هذه المشكلة لتقييم قدرة النماذج على تعلم الأنماط المعقدة وغير الخطية nonlinear decision boundary بين فئتين

يتم إنشاء بيانات (Toy Dataset) ب 200 عينة تدريبية تحتوي كل منها على ميزتين (x_0, x_1) حيث يتم سحب القيم عشوائياً من توزيع موحد (Uniform distribution) بين الرقمين -1 و 1

بعد انشاء البيانات نقوم بتعيين التسميات الحقيقية (Ground Truth Labels) لكل عينة تدريبية

$$y^{(i)} = \begin{cases} 0 & \text{if } x_0^{(i)} \times x_1^{(i)} < 0 \\ 1 & \text{otherwise} \end{cases}$$

إذا كان ناتج ضرب x_0 ب x_1 أصغر من صفر تأخذ البيانات التسمية 0
في الحالات الأخرى تأخذ التسمية 1

تقسيم البيانات :

لضمان دقة النموذج وقدرته على التعميم يتم تقسيم البيانات إلى قسمين متساويين:

- بيانات التدريب (Training): 100 عينة تستخدم لتعليم النموذج
- بيانات التحقق (Validation): 100 عينة تستخدم لاختبار أداء النموذج على بيانات لم يراها من قبل

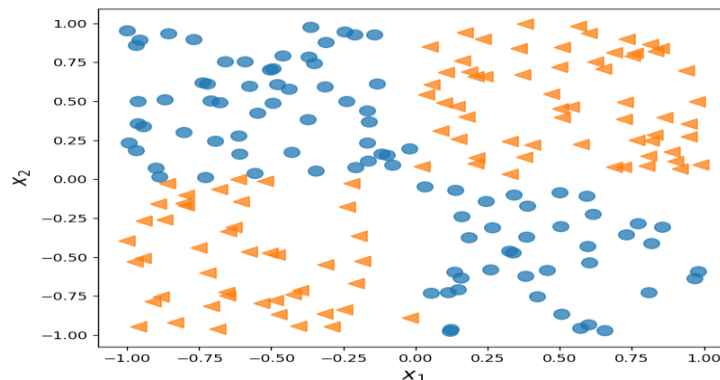
```

2 import matplotlib.pyplot as plt
3 import numpy as np
4 import torch
5
6 torch.manual_seed(1)
7 np.random.seed(1)
8 x = np.random.uniform(low=-1, high=1, size=(200, 2))
9 y = np.ones(len(x))
10 y[ x[:, 0]*x[:, 1] < 0 ] = 0
11 # Splitting data
12 n_train = 100
13 x_train = torch.tensor(x[:n_train, :], dtype=torch.float32)
14 y_train = torch.tensor(y[:n_train], dtype=torch.float32)
15 x_valid = torch.tensor(x[n_train:, :], dtype=torch.float32)
16 y_valid = torch.tensor(y[n_train:], dtype=torch.float32)
17 # Draw
18 fig = plt.figure(figsize=(6, 6))
19 plt.plot(x[y==0, 0], x[y==0, 1], 'o', alpha=0.75, markersize=10)
20 plt.plot(x[y==1, 0], x[y==1, 1], '<', alpha=0.75, markersize=10)
21 plt.xlabel(r'$x_1$', size=15)
22 plt.ylabel(r'$x_2$', size=15)
23 plt.show()

```

حيث تم تجهيز البيانات (Data Preparation)

- توليد البيانات : إنشاء 200 عينة عشوائية (x)
 - y : يتم إنشاء متجه y (يحتوي على التصنيفات) بنفس عدد العينات ب x ويقوم بتعيين كل قيمة مبدئياً إلى 1
 - $x[0, :]$ يتم اختيار العمود الأول x_0 و $x[:, 1]$ يتم اختيار العمود الثاني أي x_1 ثم يتم ضربهما ومقارنتهما مع الصفر كما تحدثنا
 - إذاً لدينا 100 عينة مصنفة ك 1 يعني كان لهما نفس الإشارة و 100 عينة مصنفة ك 0 أي اشارتين مختلفتين
 - التقسيم : تقسيم البيانات بالتساوي (100 للتدريب و 100 للتحقق) لضمان قدرة النموذج على التعميم وعدم الحفظ فقط
 - $n_train = 100$: نستخدم أول 100 نقطة للتدريب
 - $x[:n_train, :]$ يأخذ أول 100 صف من x وجميع الأعمدة
 - $x[n_train:, :]$ يأخذ باقي الصفوف (من 100 إلى 199) وجميع الأعمدة
 - التحويل لـ Tensors : تحويل البيانات من NumPy إلى PyTorch Tensors بنوع float32
- ثم نرسم :



عند اختيار بنية النموذج لمشكلة XOR يجب موازنة ما قدرة النموذج (Model Capacity) حيث تتيح زيادة العمق (عدد الطبقات) والانتساع (عدد النيورونات) للنموذج تمثيل دوال معقدة لكنها تزيد من مشكلة مخاطر الإفراط في التخصيص (Overfitting) وتجعل عملية التدريب أكثر تعقيداً لذا ينصح البدء بنموذج بسيط كمعيار أساسي ثم زيادة التعقيد تدريجياً بناءً على النتائج

- **سعة صغيرة جداً** - < النموذج غير قادر على حل المشكلة (Underfitting)
 - **سعة كبيرة جداً** - < النموذج يصعب تدريبه وقد يفشل في التعميم (Overfitting)
- تعقيباً على البدء بنموذج بسيط : مثلاً شبكة عصبية من نوع الانحدار اللوجستي (Logistic Regression) حيث يرسم خط مستقيم للفصل بين البيانات لنرى ذلك :

```
7 model = nn.Sequential(
8     nn.Linear(2, 1),
9     nn.Sigmoid()
10 )
11 print(model)
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
Sequential(
  (0): Linear(in_features=2, out_features=1, bias=True)
  (1): Sigmoid()
)
```

`nn.Sequential` يقوم بدمج الطبقات معاً يعني أن مخرجات كل طبقة تصبح تلقائياً مدخلات للطبقة التي تليها بشكل متسلسل

`nn.Linear(2, 1)`

هذه الطبقة متصلة بالكامل (Fully Connected Layer)

المدخلات (2) كل عينة تأخذ الميزتين (x_1, x_0) كما تحدثنا

المخرجات (1) قيمة وحدة ناتجة عن عملية الجمع الموزون $(\sum w_i x_i + b)$ حيث يكون خرجها هو دخل للطبقة التالية

`nn.Sigmoid()` الطبقة الثانية

تأخذ الرقم الواحد الذي خرج من الطبقة الخطية (الذي يمكن أن يكون أي قيمة مثل -5.3 أو 12.7) وتضغطه ليصبح رقم بين 0 و 1

سيساعدنا بتفسير الناتج:

إذا كان الناتج قريب من 1 يمكننا اعتباره تصنيف للفئة 1

إذا كان الناتج قريب من 0 يمكننا اعتباره تصنيف للفئة 0

إذا قمنا ببناء نموذج بطبقة واحدة فقط

تذكرة :

اختيار دالة الخسارة ليس عشوائياً وإنما يعتمد كلياً على نوع المهمة المطلوبة:

- في مشاكل التوقع (Regression): يستخدم متوسط مربعات الخطأ (Mean Square Error)
- في مشاكل التصنيف (Classification): يستخدم الإنتروبي المتقاطعة (Cross-Entropy)
- و بما أننا نحل مشكلة تصنيف ثنائي (XOR) نستخدم nn.BCELoss() اختصار لـ Binary Cross Entropy كدالة لقياس دقة النموذج

خوارزمية التحسين (Optimization Algorithm)

بعد تحديد النموذج والخسارة، نحتاج لمحسن (Optimizer) لتحديث الأوزان:

- يُعد SGD و Adam من أكثر الطرق استخدام
- قمنا بإنشاء محسن من نوع SGD وربطناه بمعاملات النموذج (model.parameters) مع تحديد معدل تعلم (Learning Rate) بسيط لنرى ذلك :

```
13 loss_fn = nn.BCELoss()
14 optimizer = torch.optim.SGD(model.parameters(), lr=0.001)
15 print(loss_fn)
16 print(optimizer)
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
BCELoss()
SGD (
Parameter Group 0
  dampening: 0
  differentiable: False
  foreach: None
  fused: None
  lr: 0.001
  maximize: False
  momentum: 0
  nesterov: False
  weight_decay: 0
)
```

The DataLoader class In PyTorch

DataLoader وظيفته مساعدتنا بتحميل البيانات للنموذج بطريقة مرتبة وفعالة خصوصاً عندما تكون مجموعة البيانات كبيرة حيث لا يمكن تحميلها كاملة في الذاكرة دفعة واحدة فالفكرة بدلاً من تمرير البيانات للنموذج مرة وحدة فالـ DataLoader يقوم بتقسيم البيانات إلى دفعات صغيرة (batches) وكل دفعة يتم تمريرها للنموذج أثناء التدريب أو الاختبار هذا يؤدي إلى تسهيل التعامل مع البيانات وتسريع عملية التدريب

حيث أن DataLoader يعتمد على dataset object حيث يمكن أن يكون مصدره:

■ مصفوفات NumPy

■ موترات بايتورث

■ ملفات CSV

■ مجلد صور

■ أو أي مصدر بيانات مخصّص يقوم المستخدم بتعريفه

حيث قبل استخدام الـ DataLoader يجب أولاً تجهيز dataset object وتحديد أي عمليات معالجة أو تحويل نحتاج إلى تطبيقها على البيانات مثل normalization أو تحويل الصور لموترات بعد تجهيز الـ dataset نقوم بإنشاء dataset object جديد ونمرّر له:

■ الـ dataset

■ حجم الدفعة (batch size)

■ وأي إعدادات إضافية حسب الحاجة

وبهذه الطريقة يصبح التعامل مع البيانات أكثر سهولة وكفاءة أثناء التدريب
لنرى الكود التالي :

```
8 from torch.utils.data import DataLoader, TensorDataset
9 # Define some sample data
10 X = torch.randn(1000, 10) # input features
11 y = torch.randint(0, 2, (1000, 1)) # binary labels
12 # Create a TensorDataset object from the data
13 dataset = TensorDataset(X, y)
14 # Create a DataLoader object with batch size 32
15 dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
16 # Iterate over the dataloader and print out some batches
17 for i, (batch_x, batch_y) in enumerate(dataloader, 1):
18     print(f"Batch {i}: input shape {batch_x.shape}, label shape {batch_y.shape}")
--
Batch 1: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 2: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 3: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 4: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 5: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 6: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 7: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 8: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 9: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 10: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 11: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 12: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 13: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 14: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 15: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 16: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 17: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 18: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 19: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 20: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 21: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 22: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 23: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 24: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 25: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 26: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 27: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 28: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 29: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 30: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 31: input shape torch.Size([32, 10]), label shape torch.Size([32, 1])
Batch 32: input shape torch.Size([8, 10]), label shape torch.Size([8, 1])
```


- TensorDataset : لدمج البيانات (المدخلات + التسميات) في كائن واحد
- DataLoader : لتحميل البيانات على شكل دفعات (batches) أثناء التدريب
- ثم نقوم بإنشاء بيانات تدريبية :
- حيث المتغير X هو بيانات الإدخال وهو مصفوفة تحتوي على 1000 عينة وكل عينة لها 10 ميزات
- والمتغير y هو التسميات (labels) وهي قيم ثنائية (0 أو 1) لـ 1000 عينة
- ثم إنشاء كائن Dataset : هنا نقوم بدمج X و y في كائن واحد TensorDataset بحيث يصبح كل عنصر في dataset عبارة عن: (input_features, label) دخل وخرج
- هنا ننشئ DataLoader ونحدد ان كل دفعة تحتوي على 32 عينة
- ثم نقوم بالمرور على الـ DataLoader دفعة دفعة
- في كل دورة:
- batch_x يحتوي على 32 صف من بيانات الإدخال
- batch_y يحتوي على 32 تسمية

1. والآن سنقوم بإنشاء data loader مع batch size = 2

لنرى ذلك :

```
train_ds = TensorDataset(x_train, y_train)
batch_size = 2
torch.manual_seed(1)
train_dl = DataLoader(train_ds, batch_size, shuffle=True)
print(train_dl)
```

- TensorDataset : يجمع بيانات التدريب (x_train) مع تسمياتها (y_train) في كائن واحد
- نحدد batch_size = 2 أي أن النموذج سيستقبل دفعتين فقط في كل خطوة تدريب
- نستخدم manual_seed لضمان أن ترتيب البيانات العشوائي سيكون نفسه في كل تشغيل
- DataLoader : يقوم بتحميل البيانات على شكل دفعات صغيرة مع خلطها في كل دورة تدريب (shuffle=True)

2. الان لنقوم بإعداد التدريب لعدد 200 (epochs): أي أن النموذج سيمر على كامل بيانات التدريب 200 مرة

لنرى ذلك :

```

31 torch.manual_seed(1)
32 num_epochs = 200
33 def train(model, num_epochs, train_dl, x_valid, y_valid):
34     loss_hist_train = [0] * num_epochs
35     accuracy_hist_train = [0] * num_epochs
36     loss_hist_valid = [0] * num_epochs
37     accuracy_hist_valid = [0] * num_epochs
38     for epoch in range(num_epochs):
39         for x_batch, y_batch in train_dl:
40             pred = model(x_batch)[: , 0]
41             loss = loss_fn(pred, y_batch)
42             loss.backward()
43             optimizer.step()
44             optimizer.zero_grad()
45             loss_hist_train[epoch] += loss.item()
46             is_correct = ((pred>=0.5).float() == y_batch).float()
47             accuracy_hist_train[epoch] += is_correct.mean()
48         loss_hist_train[epoch] /= n_train/batch_size
49         accuracy_hist_train[epoch] /= n_train/batch_size
50         pred = model(x_valid)[: , 0]
51         loss = loss_fn(pred, y_valid)
52         loss_hist_valid[epoch] = loss.item()
53         is_correct = ((pred>=0.5).float() == y_valid).float()
54         accuracy_hist_valid[epoch] += is_correct.mean()
55     return loss_hist_train, loss_hist_valid, accuracy_hist_train, accuracy_hist_valid
56
57 history = train(model, num_epochs, train_dl, x_valid, y_valid)
58
59 fig = plt.figure(figsize=(16, 4))
60 ax = fig.add_subplot(1, 2, 1)
61 plt.plot(history[0], lw=4)
62 plt.plot(history[1], lw=4)
63 plt.legend(['Train loss', 'Validation loss'], fontsize=15)
64 ax.set_xlabel('Epochs', size=15)
65 ax = fig.add_subplot(1, 2, 2)
66 plt.plot(history[2], lw=4)
67 plt.plot(history[3], lw=4)
68 plt.legend(['Train acc.', 'Validation acc.'], fontsize=15)
69 ax.set_xlabel('Epochs', size=15)

```

3. نقوم بإنشاء دالة التدريب : حيث هذه الدالة مسؤولة عن:

- تدريب النموذج على بيانات التدريب
- تقييم النموذج على بيانات التحقق (validation)
- تسجيل history الخسارة والدقة لكل حقبة
- ثم نقوم بإنشاء قوائم لحفظ ل history : لكل حقبة سيتم تخزين:
- خسارة التدريب

- دقة التدريب
- خسارة التحقق
- دقة التحقق

4. ثم لدينا حلقة التدريب : `for epoch in range(num_epochs)`

- `pred` النموذج هنا يتوقع مخرجات للدفعة الحالية
 - `loss` يتم حساب قيمة الخسارة بين التوقعات والحقائق
 - `loss.backward()` هنا يتم حساب المشتقات (backpropagation)
 - `optimizer.step()` يتم تحديث أوزان النموذج
 - `optimizer.zero_grad()` هنا يتم تصفير المشتقات قبل الدفعة التالية
- ثم حساب الدقة والخسارة لكل حقبة :

• نحسب عدد التوقعات الصحيحة في الدفعة

• نضيفها إلى مجموع دقة الحقبة

ثم تطبيع النتائج حسب عدد الدفعات :

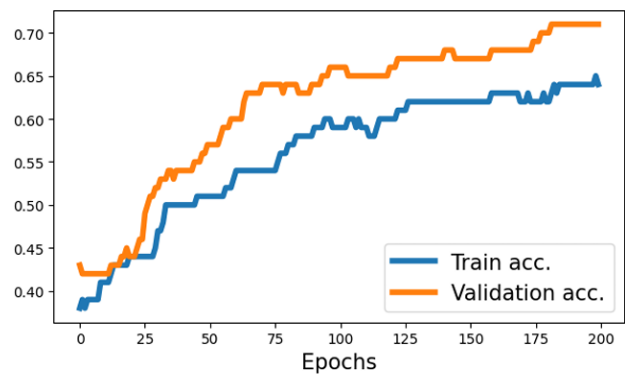
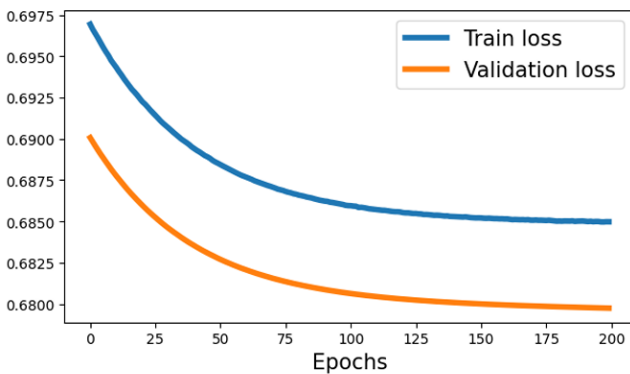
• `loss_hist_train[epoch] /= n_train/batch_size`

• `accuracy_hist_train[epoch] /= n_train/batch_size`

5. ثم تقييم النموذج على بيانات التحقق : لا يوجد تحديث للأوزان وانما فقط تقييم

6. ثم الرسم

لنرى النتيجة :



نلاحظ: ان النموذج البسيط بدون أي طبقة مخفية ينتج حد فاصل خطي فقط وهذا غير كافي لحل مشكلة XOR لأنها غير خطية اساسا

وبالتالي:

- الخسارة عالية في التدريب والتحقق
- الدقة منخفضة لأن النموذج غير قادر على التمييز بين الفئتين بشكل صحيح

النموذج المستخدم هنا عبارة عن نموذج بسيط جداً لا يحتوي على طبقات مخفية وبالتالي فهو قادر فقط على تعلم حدود قرار خطية وبما أن مشكلة XOR هي مشكلة غير خطية فإن هذا النموذج غير قادر على تمثيلها بشكل صحيح ونتيجة لذلك تظهر الخسارة مرتفعة لكل من بيانات التدريب والتحقق بينما تكون الدقة منخفضة .

حل مشكلة XOR

بناء حدود قرار غير خطية (Nonlinear Decision Boundaries) :

بما أن نقاط XOR موزعة بشكل يمنع الفصل بينها بخط مستقيم الحل الوحيد هو إضافة طبقات مخفية (Hidden Layers) مدعومة ب دوال تنشيط غير خطية (مثل ReLU أو Sigmoid) بدون هذه الدوال ستبقى الشبكة خطية مهما زاد عدد طبقاتها

نظرية التقريب العالمي (Universal Approximation Theorem)

إن الشبكة العصبية التي تحتوي على طبقة مخفية واحدة فقط ولكن بعدد كبير من الخلايا (Hidden Units) قادرة على تمثيل أي دالة رياضية مستمرة مهما كانت تعقيداتها
العرض مقابل العمق (Width vs. Depth)

هناك طريقتان لزيادة قدرة النموذج (Capacity) :

- زيادة العرض (Width) وهي إضافة عدد أكبر من الخلايا (Neurons) في نفس الطبقة المخفية
- زيادة العمق (Depth) إضافة طبقات فوق بعضها
- الميزة : النماذج العميقة توفر قدرة مماثلة للنماذج العريضة ولكن ب عدد معاملات (Parameters) أقل
- العيب : النماذج العميقة جداً تعاني من مشاكل تلاشي وانفجار الاشتقاقات (Vanishing/Exploding Gradients) عندما تصبح الشبكة عميقة جداً قد تصبح قيم الاشتقاقات (Gradients) صغيرة جداً (تتلاشى) أو ضخمة جداً (تنفجر) أثناء العودة للخلف (Backpropagation) مما يجعل تدريبها أصعب

نتائج النموذج ذو الطبقتين (Two Hidden Layers)

- عند تطبيق نموذج يحتوي على طبقتين مخفيتين (في كل منهما 4 وحدات)، نلاحظ تغيير في الأداء:
- النجاح في الفصل : استطاع النموذج أخيراً رسم منحنيات تلتف حول النقاط للفصل بين الفئات (Nonlinear Boundary)
- دقة التدريب : (100%) النموذج حفظ وفهم كل نقاط التدريب بشكل مثالي
- دقة التحقق : (95%) وجود فرق بين دقة التدريب (100%) ودقة التحقق (95%) يشير إلى وجود Overfitting صغير

```

10 model = nn.Sequential(
11     nn.Linear(2, 4),
12     nn.ReLU(),
13     nn.Linear(4, 4),
14     nn.ReLU(),
15     nn.Linear(4, 1),
16     nn.Sigmoid()
17 )
18 loss_fn = nn.BCELoss()
19 optimizer = torch.optim.SGD(model.parameters(), lr=0.015)
20 print(model)
21 print(loss_fn)
22 print(optimizer)

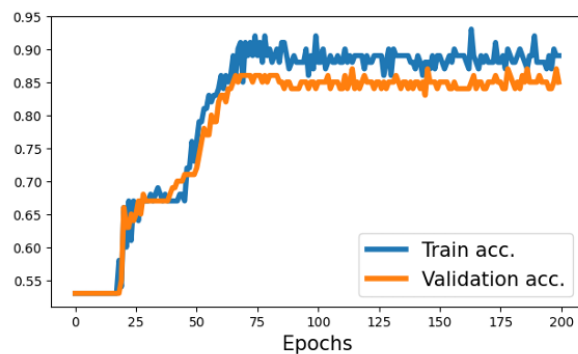
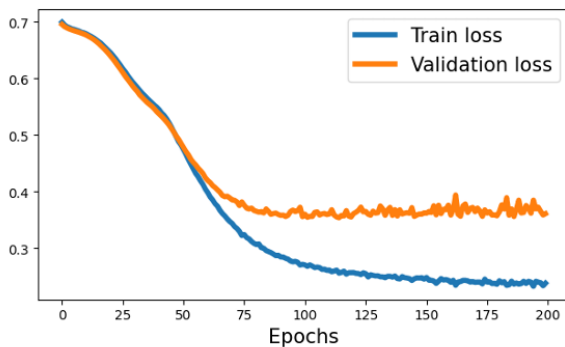
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```

Sequential(
  (0): Linear(in_features=2, out_features=4, bias=True)
  (1): ReLU()
  (2): Linear(in_features=4, out_features=4, bias=True)
  (3): ReLU()
  (4): Linear(in_features=4, out_features=1, bias=True)
  (5): Sigmoid()
)
BCELoss()
SGD (
Parameter Group 0
  dampening: 0
  differentiable: False
  foreach: None
  fused: None
  lr: 0.015
  maximize: False
  momentum: 0
  nesterov: False
  weight_decay: 0
)

```



خلاصة:

هذه التجربة اثبتت أن إضافة طبقتين مخفيتين كانت كافية لكسر قيود النماذج الخطية وحل مشكلة XOR ومع ذلك فإن الوصول لنسبة 100% في التدريب مقابل 95% في التحقق ينبهنا إلى أن النموذج بدأ يُفرط في التخصيص (Overfitting) أي أنه بدأ بحفظ تفاصيل دقيقة في بيانات التدريب قد لا تنطبق تماماً على البيانات الجديدة .

استخدام nn.Module لبناء نماذج مرنة

في الأمثلة السابقة استخدمنا nn.Sequential لبناء الشبكة وهو خيار سهل وسريع لكنه محدود لأنه يفترض أن الطبقات تتبع بعضها في خط مستقيم واحد فقط

لماذا ننتقل إلى nn.Module؟

- يعود ذلك لمرونتها يسمح لنا nn.Module لبناء نماذج معقدة تحتوي على عدة مدخلات (Inputs) أو عدة مخرجات (Outputs) أو حتى فروع جانبية (Branches) داخل الشبكة .
- يمنحك سيطرة كاملة على كيفية تدفق البيانات وتفاعل الطبقات مع بعضها .

الهيكل الأساسي للكلاس (Subclassing)

عندما ننشئ كلاس جديد يرث من nn.Module (مثل MyModule) يجب أن نعرف دالتين أساسيتين:

- دالة التهيئة `__init__()` :

هنا نقوم بتعريف الطبقات مثل nn.Linear ونخزنها كخصائص (Attributes) داخل الكلاس باستخدام `self` . في كودنا الذي سنراه استخدمنا nn.ModuleList لتخزين الطبقات في قائمة منظمة لكي يتعرف عليها بايتورث

- دالة الممر الأمامي `forward()` :

- هي الدالة تحدد كيف تدخل البيانات وتمر عبر الطبقات بالترتيب وصولاً للنتيجة النهائية

```
class MyModule(nn.Module):
    def __init__(self):
        super().__init__()
        l1 = nn.Linear(2, 4)
        a1 = nn.ReLU()
        l2 = nn.Linear(4, 4)
        a2 = nn.ReLU()
        l3 = nn.Linear(4, 1)
        a3 = nn.Sigmoid()
        l = [l1, a1, l2, a2, l3, a3]
        self.module_list = nn.ModuleList(l)
    def forward(self, x):
        for f in self.module_list:
            x = f(x)
        return x
    def predict(self, x):
        x = torch.tensor(x, dtype=torch.float32)
        pred = self.forward(x)[0]
        return (pred >= 0.5).float()

model = MyModule()
print(model)
```

```
MyModule(
  (module_list): ModuleList(
    (0): Linear(in_features=2, out_features=4, bias=True)
    (1): ReLU()
    (2): Linear(in_features=4, out_features=4, bias=True)
    (3): ReLU()
    (4): Linear(in_features=4, out_features=1, bias=True)
    (5): Sigmoid()
  )
)
```

▪ `self.module_list` :

تم تجميع الطبقات (Linear, ReLU, Sigmoid) داخل `nn.ModuleList` من اجل مساعدة بايتورش بتتبع الأوزان وتحديثها أثناء التدريب .

▪ `for f in self.module_list` :

داخل الـ `forward` قمنا بعمل حلقة تكرار تمرر البيانات `x` عبر كل طبقة بالتسلسل

▪ دالة `predict` :

من اجل تسهيل الحصول على قرار نهائي (0 أو 1) فهي تأخذ الاحتمالات الناتجة من الـ `Sigmoid` وتعتبر أن أي قيمة أكبر من أو تساوي 0.5 هي الفئة (1) .

خلاصة:

يعتبر الانتقال من `nn.Sequential` إلى `nn.Module` خطوة مهمة للانتقال من بناء النماذج الخطية البسيطة إلى هندسة الشبكات العصبية المعقدة

تعتمد بايتورش في بنيتها على كلاس `nn.Module` حيث يعتبر (Base Class) لجميع المكونات والطبقات في الشبكات العصبية بينما يوفر `nn.Sequential` وسيلة سريعة لبناء النماذج المتسلسلة يوفر `nn.Module` مرونة لتصميم بني معقدة (مثل النماذج متعددة المداخل أو متعددة المخرجات أو الفروع الجانبية) وينقل النماذج إلى الـ GPU والأهم من ذلك : طريقة استدعاء الطبقات .

الكائنات القابلة للاستدعاء (Callable Objects)

تعتمد بايتورش على ميزة في لغة Python تسمى `Callables` هذه الميزة تسمح بمعاملة كائن الطبقة (Object) كأنه دالة (Function)

فعند وراثة كلاس `nn.Module` لبناء نموذج مخصص نقوم بتعريف دالتين رئيسيتين كما تحدثنا:

1. دالة التهيئة `__init__` وفيها يتم تعريف الطبقات وتخزينها كخصائص للكائن

2. دالة المرور الامامي `forward` وفيها يتم كتابة المنطق الرياضي لتدفق البيانات

آلية الاستدعاء :

في لغة بايثون عندما نقوم باستدعاء الدوال نقوم باستدعاءها عن طريق الاقواس () مثل `model(x)` بدلاً من استدعاء `model.forward(x)` وذلك لان `nn.Module` تحتوي على دالة `__call__` ففي الواقع عندما نكتب `f(x)` تبحث بايثون عن دالة `f.__call__(x)` وتنفذها

لماذا نستخدم `model(x)` وليس `model.forward(x)` مباشرة؟

مثال صغير لفهم الفكرة : نريد مثلاً أن نذهب إلى مقهى ونطلب قهوة

الطريقة الاولى :

ذهبت الى النادل وطلبت منه قهوة ولكن طلبتها بطريقة محددة مثلاً اريد نوع حبوب القهوة ان يكون ايطالي وان تكون درجة حرارة الماء 95

هنا يكون طلبنا محدد ومباشر لانه نحنا من نخبره كيف يصنع القهوة اي تم فرض الطريقة ولكنها ممكن ان تكون غير مناسبة بالنسبة للنادل او المقهى هذا هو تماماً `model.forward(x)` .

الطريقة الثانية :

هنا فقط ذهبنا للنادل وطلبت قهوة بدون التدخل بطريقة النادل لصنعها هذا بالضبط $model(x)$ لكن ما المهمات التي يقوم بها النادل (النظام) ؟

1. يسجل طلبك : يكتب قهوة في الطلب
2. يجهز الأدوات : يأخذ الكوب ويضع الفلتر
3. ينفذ الوصفة الأساسية : يذهب ويصنع القهوة بالطريقة المثلى التي تدربوا عليها (هذا هو استدعاء forward الداخلي)
4. يضيف لمساته الخاصة : قد يقدمها مع بسكويتة (هذه هي العمليات الإضافية)
5. يتتبع كل شيء : يسجل الوقت الذي استغرقه الطلب والمكونات التي استخدمها (هذا هو بناء الرسم البياني للحساب)

فعندما نستدعي $model(x)$ نحن لا نستدعي forward فقط بل نستدعي النظام الكامل للنموذج الذي يديره `nn.Module.__call__` هذا النظام يقوم بالخطوات التالية بالترتيب:

1. تنفيذ Pre-Forward Hooks

قبل أن تبدأ أي عملية حسابية يقوم النظام بالتحقق مما إذا كان المبرمج قد وضع أي خطافات (Hooks) برمجية يجب تنفيذها قبل معالجة البيانات

- **الهدف:** تستخدم هذه الخطوة لتعديل المدخلات (Inputs) قبل دخولها للطبقات أو لتسجيل بيانات معينة لأغراض المراقبة .

2. استدعاء دالة Forward Pass

بعد الانتهاء يقوم النظام باستدعاء دالة $forward(x)$ التي قمنا بكتابتها

- **الهدف:** تنفيذ العمليات الرياضية الفعلية للطبقات (مثل الضرب في الأوزان وجمع الانحيازات)

3. معالجة Forward Hooks

بمجرد أن تنتج دالة forward مخرجاتها لا يتم إرجاع النتيجة لنا مباشرة بل يمر النظام بمرحلة الـ Hooks اللاحقة

- **الهدف:** تسمح هذه الخطوة للمكتبة (أو لك كمبرمج) بالوصول إلى مخرجات كل طبقة بشكل منفصل لتخزينها أو تحليلها دون الحاجة لتعديل كود الـ forward نفسه.

4. إدارة محرك الاشتقاق التلقائي (Autograd Management)

أثناء مرور البيانات يقوم النظام بتسجيل العمليات في رسم بياني للحسابات (Computational Graph)

- **الهدف:** هذه الخطوة لتمكين بايتورث من معرفة كيفية حساب الاشتقاقات (Gradients) لاحقاً أثناء عملية الـ Backpropagation

5. تحديث حالة النموذج (State Persistence)

يتأكد النظام من تحديث الحالة الداخلية لجميع الوحدات (Sub-modules) داخل النموذج لضمان توافق المعاملات (Parameters) مع البيانات التي مرت للتو


```

63 loss_fn = nn.BCELoss()
64 optimizer = torch.optim.SGD(model.parameters(), lr=0.015)
65 history = train(model, num_epochs, train_dl, x_valid, y_valid)

```

Visualizing Decision Boundary

بعد الانتهاء من تدريب النموذج لا يكفي الاعتماد فقط على سجل التدريب (Train History) بل نحتاج لفهم كيف يرى النموذج فضاء البيانات ويقسمه .

(a) لذا نستخدم مكتبة mlxtend :

حيث تعمل عن طريق إنشاء شبكة (Grid) من النقاط الصغيرة جداً وتطلب من النموذج عبر دالة predict() توقع الفئة لكل نقطة في هذه الشبكة ثم تلون كل منطقة بناء على هذا التوقع وهذا يوضح لنا أين يضع النموذج الخط الفاصل بين الفئة (0) والفئة (1)

(b) إضافة دالة التنبؤ predict()

لكي تتمكن المكتبة من الرسم يجب أن يكون النموذج قادر على إعطاء قرار نهائي وليس مجرد احتمال لهذا السبب قمنا بإضافة دالة predict() داخل كلاس MyModule :

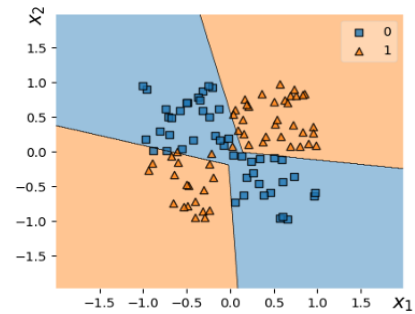
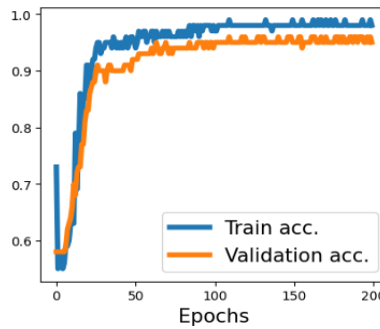
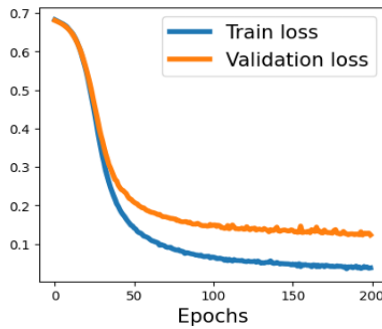
- المهمة: تحويل القيمة المستمرة الخارجة من دالة الـ Sigmoid (التي تتراوح بين 0 و 1) إلى فئات تصنيفية واضحة
- النتيجة: تعيد الدالة رقم 0 أو 1 لكل عينة وهو ما تحتاجه الخوارزمية لرسم المناطق الملونة في الرسم البياني

لنلقي نظرة على كيفية تطور أداء النموذج أثناء التدريب، وفي نفس الوقت نرى كيف يرى النموذج العالم (حدود القرار):

```

74 ax = fig.add_subplot(1, 3, 2)
75 plt.plot(history[2], lw=4)
76 plt.plot(history[3], lw=4)
77 plt.legend(['Train acc.', 'Validation acc.'], fontsize=15)
78 ax.set_xlabel('Epochs', size=15)
79
80 ax = fig.add_subplot(1, 3, 3)
81 plot_decision_regions(X=x_valid.numpy(),
82                      y=y_valid.numpy().astype(np.int64),
83                      clf=model)
84 ax.set_xlabel(r'$x_1$', size=15)
85 ax.xaxis.set_label_coords(1, -0.025)
86 ax.set_ylabel(r'$x_2$', size=15)
87 ax.yaxis.set_label_coords(-0.025, 1)
88
89 #plt.savefig('figures/13_05.png', dpi=300)
90 plt.show()

```



classifying MNIST handwritten digits تصنيف أرقام مكتوبة بخط اليد

كما تحدثنا سابقا الهدف هو تدريب نموذج ليتمكن من قراءة صور لأرقام مكتوبة بخط اليد (من 0 إلى 9) والتعرف عليها بدقة

ما هي مكتبة torchvision؟

هي مكتبة تابعة لبايثون متخصصة في الرؤية الحاسوبية (Computer Vision) توفر لنا مجموعات بيانات جاهزة (Datasets) وأدوات لتحويل الصور (Transforms) وتصاميم لنماذج شهيرة .

1. تعتبر مرحلة إعداد البيانات (Setup) هي الركيزة الأساسية في تصنيف أرقام MNIST

قمنا باستخدام مكتبة torchvision لتحميل البيانات وتطبيق عمليات التحويل (Transforms) لتحويل الصور إلى تنسيقات رقمية (Tensors) مناسبة للنموذج كما تم تنظيم البيانات باستخدام DataLoader بتقسيمها إلى دفعات (Mini-batches) بحجم 64 عينة مع تفعيل خاصية الخلط العشوائي (Shuffle) وذلك لضمان كفاءة استهلاك الذاكرة وتعزيز قدرة النموذج على التعميم (Generalization) بعيداً عن حفظ ترتيب البيانات .

لنرى تنفيذ ذلك:

```
import torchvision
from torchvision import transforms

image_path = './'
transform = transforms.Compose([transforms.ToTensor()])

mnist_train_dataset = torchvision.datasets.MNIST(root=image_path, train=True,
                                                transform=transform, download=True)
mnist_test_dataset = torchvision.datasets.MNIST(root=image_path, train=False,
                                                transform=transform, download=False)

batch_size = 64
torch.manual_seed(1)
train_dl = DataLoader(mnist_train_dataset, batch_size, shuffle=True)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 18.8MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 488kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.48MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 12.5MB/s]
```

The Setup Step

تتضمن هذه المرحلة ثلاث عمليات أساسية:

1. أولاً: تحويل البيانات (transforms.ToTensor)

الصور في الأصل تكون بصيغة (Pixels) بقيم تتراوح بين 0 و 255 وظيفه ToTensor() هي:

- تحويل الصورة إلى موتر
- تغيير نطاق القيم (Scaling) لتصبح بين 0 و 1 من اجل تسريع عملية التعلم

2. ثانياً: تحميل البيانات (MNIST Dataset)

نستخدم torchvision.datasets.MNIST لجلب البيانات:

■ train=True :

لتحميل 60,000 صورة مخصصة لتدريب النموذج

■ train=False :

لتحميل 10,000 صورة مخصصة لاختبار النموذج (بيانات لم يرها النموذج من قبل).

■ download=True :

تحميل البيانات من الإنترنت إذا لم تكن موجودة في الجهاز

3. ثالثاً: تحميل البيانات (DataLoader)

■ batch_size = 64 :

بدلاً من إدخال 60 ألف صورة دفعة واحدة (مما قد يسبب انفجاراً في الذاكرة) نقوم بإدخالها في مجموعات صغيرة كل

مجموعة تحتوي على 64 صورة

■ shuffle=True :

إعادة ترتيب الصور عشوائياً في كل دورة تدريبية لضمان أن الموديل لا يحفظ ترتيب الأرقام بل يتعلم شكلها

2. معالجة البيانات (Data Preprocessing)

قبل إدخال الصور إلى النموذج يجب التأكد من أنها في صيغة رياضية مناسبة

- تحويل البيانات (ToTensor) استخدمنا transforms.Compose للتطبيق دالة ToTensor() هذه الدالة تقوم بمهمتين كما تحدثنا :

1. تحويل بكسلات الصورة من أرقام صحيحة (Integers) إلى أرقام عشرية (Floating point tensors)

2. التطبيع (Normalization): تحويل قيم البكسلات من النطاق [0, 255] إلى النطاق [0, 1] من اجل

استقرار تدريب الشبكة العصبية وسرعة تقاربها (Convergence)

- التسميات (Labels): بما أن المخرجات هي أرقام صحيحة من 0 إلى 9 (تمثل الأرقام العشرة) فهي جاهزة للاستخدام مباشرة في تصنيف البيانات ولا تحتاج لأي معالجة إضافية .

3. بناء نموذج الشبكة العصبية (Constructing the NN Model)
تم بناء نموذج متصل بالكامل (Fully Connected) يتكون من عدة طبقات

```
hidden_units = [32, 16]
image_size = mnist_train_dataset[0][0].shape
input_size = image_size[0] * image_size[1] * image_size[2]

all_layers = [nn.Flatten()]
for hidden_unit in hidden_units:
    layer = nn.Linear(input_size, hidden_unit)
    all_layers.append(layer)
    all_layers.append(nn.ReLU())
    input_size = hidden_unit

all_layers.append(nn.Linear(hidden_units[-1], 10))
model = nn.Sequential(*all_layers)

print(model)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=32, bias=True)
  (2): ReLU()
  (3): Linear(in_features=32, out_features=16, bias=True)
  (4): ReLU()
  (5): Linear(in_features=16, out_features=10, bias=True)
)
```

(a) مرحلة (nn.Flatten)

كما نعلم ان الصور في MNIST تكون بأبعاد [1, 28, 28] (Channel واحدة، طول 28، وعرض 28) وهي باللون الأبيض والأسود ولكن الطبقات الخطية (nn.Linear) لا تقبل مصفوفات ثلاثية الأبعاد لذا نستخدم nn.Flatten() لتحويل الصورة إلى متجه أحادي البعد (1D Vector) بطول 784 بكسل ($1 * 28 * 28 = 784$)

(b) الطبقات المخفية (Hidden Layers)

- حلقة for لتكرار بناء الطبقات بناء على قائمة [32, 16]:
- الطبقة الأولى: تحول المدخلات من 784 إلى 32 وحدة (Unit)
 - الطبقة الثانية: تحول المخرجات من 32 إلى 16 وحدة

- استخدمنا دالة التنشيط ReLU بعد كل طبقة خطية لإضافة غير الخطية للنموذج مما يمكنه من تعلم أنماط معقدة في أشكال الأرقام .

(c) طبقة المخرجات (Output Layer)

تنتهي الشبكة بطبقة خطية أخيرة تحتوي على 10 وحدات حيث تمثل كل وحدة احتمالية انتماء الصورة لواحد من الأرقام (0-9)

إذاً: تبدأ عملية بناء النموذج بمرحلة معالجة حيث يتم تطبيع بكسلات الصور لتصبح ضمن النطاق [0, 1] لضمان استقرار العمليات الحسابية أما من الناحية الهيكلية فقد تم تصميم شبكة عصبية تعتمد على تحويل البيانات من شكلها ثلاثي الأبعاد إلى متجه خطي عبر طبقة nn.Flatten يتكون النموذج من طبقتين مخفيتين (32 و 16 وحدة) مع استخدام دالة التنشيط ReLU لتعزيز قدرة التعلم وصولاً إلى طبقة المخرجات المكونة من 10 وحدات هذا التصميم يوازن بين الكفاءة الحسابية والقدرة على استخلاص الميزات المعقدة من الأرقام المكتوبة بخط اليد

4. نستخدم النموذج للتدريب والتقييم والتنبؤ :

هنا يبدأ النموذج بالتعلم فعلياً من البيانات التي جهرناها سابقاً

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
torch.manual_seed(1)
num_epochs = 20
for epoch in range(num_epochs):
    accuracy_hist_train = 0
    for x_batch, y_batch in train_dl:
        pred = model(x_batch)
        loss = loss_fn(pred, y_batch)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        is_correct = (torch.argmax(pred, dim=1) == y_batch).float()
        accuracy_hist_train += is_correct.sum()
    accuracy_hist_train /= len(train_dl.dataset)
    print(f'Epoch {epoch} Accuracy {accuracy_hist_train:.4f}')
```

```
Epoch 0 Accuracy 0.8531
Epoch 1 Accuracy 0.9287
Epoch 2 Accuracy 0.9413
Epoch 3 Accuracy 0.9506
Epoch 4 Accuracy 0.9558
Epoch 5 Accuracy 0.9592
Epoch 6 Accuracy 0.9627
Epoch 7 Accuracy 0.9649
Epoch 8 Accuracy 0.9673
Epoch 9 Accuracy 0.9690
Epoch 10 Accuracy 0.9711
Epoch 11 Accuracy 0.9729
Epoch 12 Accuracy 0.9737
Epoch 13 Accuracy 0.9747
Epoch 14 Accuracy 0.9766
Epoch 15 Accuracy 0.9778
Epoch 16 Accuracy 0.9780
Epoch 17 Accuracy 0.9798
Epoch 18 Accuracy 0.9806
Epoch 19 Accuracy 0.9816
```

1. Loss & Optimizer

- **دالة الخسارة: (CrossEntropyLoss)** تستخدم هذه الدالة لأننا بصدد مشكلة تصنيف متعدد (Multiclass Classification) لعشرة أرقام هي تقيس مدى بُعد توقعات النموذج عن الأرقام الحقيقية
- **المُحسن: (Adam Optimizer)** اخترنا خوارزمية Adam لأنها ذكية وسريعة في ضبط الأوزان (Learning Rate Adaptation) مقارنة بـ SGD التقليدي

2. دورة التدريب (Training Loop)

- تتم عملية التدريب عبر 20 دورة (Epochs) ، وفي كل دورة نقوم بالآتي:
1. **المرور الأمامي: (Forward Pass)** نمرر دفعة من الصور لـ `model(x_batch)` للحصول على التوقعات
 2. **حساب الخسارة:** نقارن التوقعات بالنتائج الحقيقية عبر `loss_fn`
 3. **المرور الخلفي: (Backward Pass)** نستخدم `loss.backward()` لحساب الاشتقاق التي تخبرنا بكيفية تعديل الأوزان لتقليل الخطأ
 4. **تحديث الأوزان:** يقوم `optimizer.step()` بتعديل أوزان الشبكة فعلياً ثم نصفر الاشتقاقات (`zero_grad`) من أجل الدفعة التالي .

3. تقييم الأداء (Evaluation)

- **دقة التدريب (Train Accuracy):** وصل النموذج إلى دقة 96.3% وهذا يدل على أن الشبكة استطاعت استيعاب الأنماط المختلفة للأرقام المكتوبة بخط اليد
- **اختبار النموذج (Testing):** بعد التدريب نختبر النموذج على بيانات لم يرها أبداً (`mnist_test_dataset`) حيث أننا قمنا بقسمة البيانات على 255 يدوياً لضمان أن تكون المدخلات في النطاق [0, 1] تماماً كما حدث أثناء التدريب .

```
pred = model(mnist_test_dataset.data / 255.)
is_correct = (torch.argmax(pred, dim=1) == mnist_test_dataset.targets).float()
print(f'Test accuracy: {is_correct.mean():.4f}')
```

إذاً: تمثل مرحلة التدريب عملية التحسين المستمر لمعاملات النموذج باستخدام دالة الخسارة `CrossEntropyLoss` وخوارزمية Adam للتحسين حيث خضع النموذج لـ 20 دورة تدريبية مكثفة حيث يتم تحديث الأوزان بناءً على إشارات الخطأ الناتجة عن المرور الخلفي (Backpropagation) أظهرت النتائج كفاءة عالية بوصول دقة التدريب إلى 96.3% ولضمان قدرة النموذج على التعميم تم تقييمه باستخدام بيانات الاختبار المستقلة بعد إجراء عملية التطبيع (Normalization) اللازمة مما يثبت نجاح الشبكة العصبية في التنبؤ بالأرقام المكتوبة بخط اليد حتى في الحالات التي لم يسبق لها رؤيتها .

تسريع الأجهزة (Hardware Acceleration) + باستخدام GPU

تعتمد قوة بايتورش الحقيقية على قدرتها على نقل الحسابات المعقدة من المعالج المركزي (CPU) إلى وحدات معالجة الرسومات (GPU) أو مسرعات أخرى، مما يسرع عملية التدريب بمئات الأضعاف

1. آلية العمل في PyTorch

يمكن نسخ الـ Tensors (مصفوفات البيانات) بسهولة إلى الـ GPU إذا كنت تستخدم منصة مثل Google Colab فإن العملية بسيطة جداً حيث يتم توفير كل شيء مسبقاً (مكتبات CUDA و cuDNN) بمجرد تغيير نوع الـ Runtime إلى GPU

2. المسرعات المدعومة (Supported Accelerators)

لا يقتصر دعم بايتورش على نوع واحد من الأجهزة بل يمتد ليشمل منصات متنوعة:

▪ Nvidia GPUs (CUDA)

الخيار الأكثر شيوعاً حيث يستخدم مكتبات CUDA و cuDNN لتسريع العمليات الحسابية بشكل هائل

▪ Apple Silicon (MPS)

تدعم أجهزة الماك الحديثة (رقاقات M1 ، M2) وما بعدها من خلال Metal Performance Shaders مما يسمح لمستخدمي آبل بتدريب النماذج محلياً بكفاءة عالية

▪ AMD (ROCm)

يدعم معالجات AMD Instinct و Radeon عبر منصة ROCm أو عبر DirectML في بيئة ويندوز

▪ Intel (oneAPI)

يدعم معالجات وجرافيك إنتل على أنظمة لينكس وويندوز

▪ Google TPUs

وحدات مخصصة من غوغل للتعامل مع الموترات الضخمة وتدريب النماذج العملاقة ويتم التعامل معها عبر مكتبة torch_xla

إدارة الأجهزة في بايتورش (Device Management)

1. آلية التحقق التلقائي (Fallback Mechanism)

```
--
16 if torch.cuda.is_available():
17     device = "cuda"
18 elif torch.backends.mps.is_available():
19     device = "mps"
20 else:
21     device = "cpu"
22
23 print(device)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTEI

cpu

نقوم بعملية ترتيب أولويات (Prioritization):

- أولاً يبحث عن CUDA لأجهزة (Nvidia)
- إذا لم يجد يبحث عن MPS لأجهزة (Apple Silicon)
- إذا لم يجد أيّاً منهما يعود للخيار المضمون وهو الـ CPU

2. كيف ننقل البيانات إلى المسرّع؟

هناك طريقتان أساسيتان للقيام بذلك:

- الطريقة الأولى النسخ Copying إنشاء الموتر (Tensor) على الـ CPU أولاً (وهو الوضع الافتراضي) ثم نقله باستخدام دالة (to(device)). هذه الطريقة مفيدة عند تحميل البيانات من قرص التخزين ثم دفعها للمعالج الرسومي
- الطريقة الثانية (الإنشاء المباشر Direct Creation): إنشاء الموتر مباشرة داخل ذاكرة المسرّع باستخدام الوسيط device=device هذه الطريقة أسرع لأنها توفر وقت النقل بين الذاكرتين

```
25 M = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
26 M = M.to(device)
27 M.device
28 #Alternatively, we can create the tensor directly on the GPU using the device argument:
29 M = torch.tensor([[1., 2., 3.], [4., 5., 6.]], device=device)
30 print(M)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITLENS

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

- ملاحظة: إذا كان لديك أكثر من كرت شاشة من نوع يمكنك تحديد الكرت المطلوب برقم المعرف الخاص به مثل "cuda:0" للكرت الأول أو "cuda:1" للثاني

كفاءة الحسابات داخل المسرّع (GPU Computing Efficiency)

بمجرد نقل البيانات (Tensors) إلى وحدة معالجة الرسوميات (GPU) يتغير سلوك النظام ليصبح أكثر سرعة وذكاء في التعامل مع العمليات الحسابية -التنفيذ التلقائي على نفس الجهاز: عندما نقوم بإجراء عملية رياضية (مثل ضرب المصفوفات $M @ M.T$) على موتر موجود في الـ GPU فإن بايتورش ينفذ هذه العملية بالكامل داخل ذاكرة ومعالج الـ GPU

```
31 R = M @ M.T
32 print(R)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL

```
tensor([[14., 32.],
        [32., 77.]])
```

■ ملاحظة:

- إذا كان M على الـ GPU فإن R سيكون دائماً على الـ GPU
- إذا حاولت جمع موتر من الـ CPU مع موتر من الـ GPU سيعطيك بايتورث خطأ (RuntimeError) لأنه يشترط وجود جميع العناصر في نفس المكان لإتمام العملية

مقارنة الأداء: المعالج المركزي (CPU) مقابل المعالج الرسومي (GPU)

تعتمد كفاءة تسريع الـ GPU على حجم الحسابات وتوازي العمليات حيث أظهرت التجارب أن الـ GPU يتفوق بشكل كبير في المصفوفات الضخمة بفضل قدرته على المعالجة المتوازية بينما تتضاءل هذه الفائدة في المهام الصغيرة بسبب تكلفة نقل البيانات التي تصبح عائقاً (Bottleneck) لذا فإن اختيار المعالج المناسب يعتمد على التوازن بين تعقيد النموذج وحجم البيانات حيث تبرز قوة الـ GPU الحقيقية في النماذج العميقة التي تتطلب قدرة حسابية هائلة وتدفق بيانات مستمر عبر تقنيات مثل CUDA و cuDNN

إذاً :

مفهوم التوازي (Parallelism)

يعمل الـ GPU من خلال تقسيم العمليات الحسابية الضخمة إلى آلاف العمليات الصغيرة جداً وتشغيلها في وقت واحد (بالتوازي) عبر آلاف النواة (Cores) بينما يحتوي الـ CPU على عدد قليل من النوى القوية المصممة للمهام التسلسلية المعقدة .

متى يتفوق الـ GPU ؟

- في Compute Heavy :

عندما تكون المصفوفات كبيرة (مثل 1000×1000) تظهر قوة الـ GPU بوضوح

```
M = torch.rand((1000, 1000)) # on the CPU
%timeit M @ M.T
```

26.5 ms ± 3.94 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

```
M = torch.rand((1000, 1000), device="cuda") # on the GPU
%timeit M @ M.T
```

620 µs ± 15 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)

في هذا المثال حقق الـ GPU سرعة أعلى بـ 29 ضعف مقارنة بالـ CPU

- في النماذج العميقة: النماذج التي تحتوي على طبقات كثيرة ومعاملات ضخمة تستفيد أقصى استفادة من سرعة الـ GPU وسعة ذاكرته (RAM)

مشكلة Bottleneck وفي حال كانت المصفوفات صغيرة

- في المصفوفات الصغيرة: إذا كانت المصفوفة صغيرة (مثلاً $100 * 100$) تنخفض نسبة التسريع بشكل كبير (لتصل إلى ضعفين فقط أو أقل)
- لماذا؟ لأن الوقت المستغرق في تجهيز المهمة ونقل البيانات إلى الـ GPU قد يستغرق وقتاً أطول من الحساب نفسه ففي المهام المتناهية الصغر قد يكون الـ CPU أسرع لأنه لا يحتاج لعناء نقل البيانات
- Bottleneck
- تنتج كثرة نقل البيانات بين المعالج المركزي (CPU) والمعالج الرسومي (GPU)
- لماذا؟ لأن سرعة نقل البيانات عبر ممرات اللوحة الأم (Bus) أبطأ بكثير من سرعة الحسابات داخل الـ GPU نفسه.
- الحل: إبقاء البيانات والنتائج الوسيطة داخل الـ GPU طوال فترة التدريب مما يجعل تدريب النماذج العميقة ممكن وسريع

أدوات القياس والتقنيات المدعومة

- %timeit : أداة سحرية في بيئات Jupyter و Colab لقياس متوسط وقت تنفيذ الكود بدقة
- CUDA & cuDNN : التقنيات التي تجعل هذا التسريع ممكناً على أجهزة Nvidia حيث توفر مكتبة برمجية محسنة خصيصاً لتسريع طبقات الشبكات العصبية العميقة.
- ملاحظة: أن اختيار الـ GPU المناسب يعتمد على حجم الـ RAM المتوفر فيه فكلما زاد حجم النموذج احتجت لـ RAM أعلى لتجنب خطأ Out of Memory

النهاية